

Session 3

Arrays, Pointers & Bitwise Operations



Smooth a noisy sensor, give a box a second name, pack eight answers into a byte.

Part	Topic	Time
01	References: a second name for the same box	5 min
02	Pointers: the address of a box	7 min
03	Bitwise operators: eight switches in one byte	9 min
04	Practice: a moving average of the light sensor	9 min

You know arrays from Lesson 4.

Idea	Where
Arrays: create, write, read, index from 0	Lesson 4 · Array & for loop
Average and largest value with a for loop	Lesson 4 · Array & for loop
Writing functions with parameters and return	Session 2 · Control structures & functions

Eight words you will hear today.

Word	Plain meaning
Reference	A second name for a variable that already exists. Change one, and you change both.
Address	Where a variable lives in memory — like a locker number.
Pointer	A variable that holds an address.
Bit	The smallest piece of data: 0 or 1. A byte is 8 bits.
Binary	Writing a number in bits, like 101 for 5.
Bitwise operator	An operator that works on each bit separately: &, , ^, <<, >>.
Moving average	The average of only the last few readings, updated as each new one arrives.
Modulo (%)	The remainder after dividing: 10 % 10 is 0, 7 % 10 is 7.

A reference is a nickname: two names, one box.

At school you are "Alex"; at home you are "Al". If Al gets a haircut, so does Alex.

In Session 2 a function got a **copy** of each value. Add & after the type, and it gets the **real variable** instead.

```
void addBonus(int &score)
```

Passing by reference.

```
1 void addBonus(int &score) {  
2     score = score + 10;  
3 }  
4  
5 void setup() {  
6     Serial.begin(9600);  
7     int points = 50;  
8     addBonus(points);  
9     Serial.println(points);  
10 }
```

LINE 6 · STEP 1 OF 5

Start the Serial monitor.

VARIABLES

None yet.

SERIAL MONITOR

Passing by reference.

```
1 void addBonus(int &score) {  
2     score = score + 10;  
3 }  
4  
5 void setup() {  
6     Serial.begin(9600);  
7     int points = 50;  
8     addBonus(points);  
9     Serial.println(points);  
10 }
```

LINE 7 · STEP 2 OF 5

points starts at 50.

VARIABLES

points = 50

SERIAL MONITOR

Passing by reference.

```
1 void addBonus(int &score) {  
2     score = score + 10;  
3 }  
4  
5 void setup() {  
6     Serial.begin(9600);  
7     int points = 50;  
8     addBonus(points);  
9     Serial.println(points);  
10 }
```

LINE 8 · STEP 3 OF 5

Call addBonus. Because of &, score is not a copy — it is another name for points.

VARIABLES

points = 50

score = 50 (same box)

SERIAL MONITOR

Passing by reference.

```
1 void addBonus(int &score) {  
2     score = score + 10;  
3 }  
4  
5 void setup() {  
6     Serial.begin(9600);  
7     int points = 50;  
8     addBonus(points);  
9     Serial.println(points);  
10 }
```

LINE 2 · STEP 4 OF 5

Add 10 to score... which is points.

VARIABLES

points = 50 → 60

score = 50 → 60

SERIAL MONITOR

Passing by reference.

```
1 void addBonus(int &score) {  
2     score = score + 10;  
3 }  
4  
5 void setup() {  
6     Serial.begin(9600);  
7     int points = 50;  
8     addBonus(points);  
9     Serial.println(points);  
10 }
```

LINE 9 · STEP 5 OF 5

Back in `setup()`, `points` really changed. Without the `&`, it would print 50.

VARIABLES

`points = 50 → 60`

SERIAL MONITOR

60

A pointer is a locker number on a sticky note.

The locker holds your stuff (the value). The sticky note holds the locker's number (the address). With the note, you can go to the locker and change what is inside.

Symbol	Say it as	Example
<code>&x</code>	"the address of x"	<code>int *p = &light;</code>
<code>int *p</code>	"p is a pointer to an int"	—
<code>*p</code>	"go to the address in p"	<code>*p = 600;</code>

Changing a box through its address.

```
1 int light = 512;  
2 int *p = &light;  
3 *p = 600;  
4 Serial.println(light);
```

LINE 1 · STEP 1 OF 4

A normal int box holding 512.

VARIABLES

light = 512

SERIAL MONITOR

Changing a box through its address.

```
1 int light = 512;
2 int *p = &light;
3 *p = 600;
4 Serial.println(light);
```

LINE 2 · STEP 2 OF 4

`&light` is the address of `light`. `p` keeps it — `p` "points to" `light`.

VARIABLES

`light = 512`

`p = address of light`

SERIAL MONITOR

Changing a box through its address.

```
1 int light = 512;
2 int *p = &light;
3 *p = 600;
4 Serial.println(light);
```

LINE 3 · STEP 3 OF 4

*p means "the box p points to". Put 600 in it.

VARIABLES

light = 512 → 600

p = address of light

SERIAL MONITOR

Changing a box through its address.

```
1 int light = 512;  
2 int *p = &light;  
3 *p = 600;  
4 Serial.println(light);
```

LINE 4 · STEP 4 OF 4

light itself changed, though line 3 never named it.

VARIABLES

light = 512 → 600

p = address of light

SERIAL MONITOR

600

An array's name works like a pointer to its first box.

That is why a function can take a whole array — `float values[]` — and read every reading without copying all ten. You will use this in the practice.

On the INCIPE Board: in firmware, pointers also reach the chip's hardware directly. Here the runtime does that for you — your code calls the `incipe.*` helpers — so you use pointers on your own variables and arrays.

A byte is a row of 8 light switches.

Each bit is a switch: on (1) or off (0). One `uint8_t` can hold eight yes/no answers. Counting from the right, the switches are numbered 0 to 7.

Switch	7	6	5	4	3	2	1	0
Worth	128	64	32	16	8	4	2	1

So binary **101** is switch 2 and switch 0 on: $4 + 1 = 5$.

Each one works on every bit separately.

Op	Name	Each bit is 1 when...	Example	Result
&	AND	both bits are 1	1100 & 1010 (12 & 10)	1000 = 8
	OR	either bit is 1	1100 1010 (12 10)	1110 = 14
^	XOR	the bits are different	1100 ^ 1010 (12 ^ 10)	0110 = 6
<<	shift left	every bit moves left	0001 << 3 (1 << 3)	1000 = 8
>>	shift right	every bit moves right	1000 >> 2 (8 >> 2)	0010 = 2

Not the same as && and || from Lesson 2: those compare whole true/false values. & and | work on every bit.

Eight flags in one byte.

```
1  uint8_t flags = 0;
2  flags = flags | (1 << 0);
3  flags = flags | (1 << 2);
4  Serial.println(flags, BIN);
5  bool pressed = flags & (1 << 2);
6  Serial.println(pressed);
7  flags = flags ^ (1 << 0);
8  Serial.println(flags, BIN);
```

LINE 1 · STEP 1 OF 8

One byte, all eight switches off.

VARIABLES

flags = 00000000

SERIAL MONITOR

Eight flags in one byte.

```
1  uint8_t flags = 0;
2  flags = flags | (1 << 0);
3  flags = flags | (1 << 2);
4  Serial.println(flags, BIN);
5  bool pressed = flags & (1 << 2);
6  Serial.println(pressed);
7  flags = flags ^ (1 << 0);
8  Serial.println(flags, BIN);
```

LINE 2 · STEP 2 OF 8

1 << 0 is 00000001. OR it in: switch 0 on — "sensor ready".

VARIABLES

flags = 00000000 → 00000001

SERIAL MONITOR



Eight flags in one byte.

```
1  uint8_t flags = 0;
2  flags = flags | (1 << 0);
3  flags = flags | (1 << 2);
4  Serial.println(flags, BIN);
5  bool pressed = flags & (1 << 2);
6  Serial.println(pressed);
7  flags = flags ^ (1 << 0);
8  Serial.println(flags, BIN);
```

LINE 3 · STEP 3 OF 8

1 << 2 is 00000100. OR it in: switch 2 on — "button pressed".

VARIABLES

flags = 00000001 → 00000101

SERIAL MONITOR

Eight flags in one byte.

```
1  uint8_t flags = 0;
2  flags = flags | (1 << 0);
3  flags = flags | (1 << 2);
4  Serial.println(flags, BIN);
5  bool pressed = flags & (1 << 2);
6  Serial.println(pressed);
7  flags = flags ^ (1 << 0);
8  Serial.println(flags, BIN);
```

LINE 4 · STEP 4 OF 8

BIN prints the number in binary. Leading zeros are not printed.

VARIABLES

flags = 00000101

SERIAL MONITOR

101

Eight flags in one byte.

```
1  uint8_t flags = 0;
2  flags = flags | (1 << 0);
3  flags = flags | (1 << 2);
4  Serial.println(flags, BIN);
5  bool pressed = flags & (1 << 2);
6  Serial.println(pressed);
7  flags = flags ^ (1 << 0);
8  Serial.println(flags, BIN);
```

LINE 5 · STEP 5 OF 8

AND with 00000100 keeps only switch 2. It is on, so the result is not 0: true.

VARIABLES

flags = 00000101

pressed = true

SERIAL MONITOR

101

Eight flags in one byte.

```
1  uint8_t flags = 0;
2  flags = flags | (1 << 0);
3  flags = flags | (1 << 2);
4  Serial.println(flags, BIN);
5  bool pressed = flags & (1 << 2);
6  Serial.println(pressed);
7  flags = flags ^ (1 << 0);
8  Serial.println(flags, BIN);
```

LINE 6 · STEP 6 OF 8

A bool prints as 1 for true.

VARIABLES

flags = 00000101

pressed = true

SERIAL MONITOR

101

1

Eight flags in one byte.

```
1  uint8_t flags = 0;
2  flags = flags | (1 << 0);
3  flags = flags | (1 << 2);
4  Serial.println(flags, BIN);
5  bool pressed = flags & (1 << 2);
6  Serial.println(pressed);
7  flags = flags ^ (1 << 0);
8  Serial.println(flags, BIN);
```

LINE 7 · STEP 7 OF 8

XOR flips switch 0: on → off.

VARIABLES

flags = 00000101 → 00000100

pressed = true

SERIAL MONITOR

```
101
1
```

Eight flags in one byte.

```
1  uint8_t flags = 0;
2  flags = flags | (1 << 0);
3  flags = flags | (1 << 2);
4  Serial.println(flags, BIN);
5  bool pressed = flags & (1 << 2);
6  Serial.println(pressed);
7  flags = flags ^ (1 << 0);
8  Serial.println(flags, BIN);
```

LINE 8 · STEP 8 OF 8

Only switch 2 is left. To force a bit off: flags & ~(1 << n).

VARIABLES

flags = 00000100

pressed = true

SERIAL MONITOR

```
101
1
100
```

Meet the light sensor.

ONE LINE OF CODE

```
float light = incipe.getLightIntensity();
```

Item	Detail
Value	A float from 0 (dark) to 1023 (bright)
Not detected	-1
Read it every	20–500 ms — light changes quickly

Full details: [Wiki › Light sensor](#).

Judge a team by its last 10 games.

Not its whole history, and not just yesterday. Each new reading pushes out the oldest, so the number follows real changes but ignores single flickers.

The average of an array.

```
1 float average(float values[], int count) {  
2     float total = 0;  
3     for (int i = 0; i < count; i++) {  
4         total = total + values[i];  
5     }  
6     return total / count;  
7 }
```

LINE 1 · STEP 1 OF 10

`values[]` is passed like a pointer: nothing is copied.
Say it holds {10, 20, 30}.

VARIABLES

`values` = {10, 20, 30}
`count` = 3

HANDS BACK

The average of an array.

```
1 float average(float values[], int count) {  
2     float total = 0;  
3     for (int i = 0; i < count; i++) {  
4         total = total + values[i];  
5     }  
6     return total / count;  
7 }
```

LINE 2 · STEP 2 OF 10

Start the total at 0.

VARIABLES

values = {10, 20, 30}

count = 3

total = 0

HANDS BACK

The average of an array.

```
1 float average(float values[], int count) {  
2     float total = 0;  
3     for (int i = 0; i < count; i++) {  
4         total = total + values[i];  
5     }  
6     return total / count;  
7 }
```

LINE 3 · STEP 3 OF 10

i starts at 0; $0 < 3$, so go in.

VARIABLES

values = {10, 20, 30}

count = 3

total = 0

i = 0

HANDS BACK

The average of an array.

```
1 float average(float values[], int count) {  
2     float total = 0;  
3     for (int i = 0; i < count; i++) {  
4         total = total + values[i];  
5     }  
6     return total / count;  
7 }
```

LINE 4 · STEP 4 OF 10

Add values[0] = 10.

VARIABLES

values = {10, 20, 30}

count = 3

total = 0 → 10

i = 0

HANDS BACK

The average of an array.

```
1 float average(float values[], int count) {  
2     float total = 0;  
3     for (int i = 0; i < count; i++) {  
4         total = total + values[i];  
5     }  
6     return total / count;  
7 }
```

LINE 3 · STEP 5 OF 10

i becomes 1; $1 < 3$.

VARIABLES

values = {10, 20, 30}

count = 3

total = 0 → 10

i = 0 → 1

HANDS BACK

The average of an array.

```
1 float average(float values[], int count) {  
2     float total = 0;  
3     for (int i = 0; i < count; i++) {  
4         total = total + values[i];  
5     }  
6     return total / count;  
7 }
```

LINE 4 · STEP 6 OF 10

Add values[1] = 20.

VARIABLES

values = {10, 20, 30}

count = 3

total = 0 → 10 → 30

i = 0 → 1

HANDS BACK

The average of an array.

```
1 float average(float values[], int count) {  
2     float total = 0;  
3     for (int i = 0; i < count; i++) {  
4         total = total + values[i];  
5     }  
6     return total / count;  
7 }
```

LINE 3 · STEP 7 OF 10

i becomes 2; $2 < 3$.

VARIABLES

values = {10, 20, 30}

count = 3

total = 0 → 10 → 30

i = 0 → 1 → 2

HANDS BACK

The average of an array.

```
1 float average(float values[], int count) {  
2     float total = 0;  
3     for (int i = 0; i < count; i++) {  
4         total = total + values[i];  
5     }  
6     return total / count;  
7 }
```

LINE 4 · STEP 8 OF 10

Add values[2] = 30.

VARIABLES

values = {10, 20, 30}

count = 3

total = 0 → 10 → 30 → 60

i = 0 → 1 → 2

HANDS BACK

The average of an array.

```
1 float average(float values[], int count) {  
2     float total = 0;  
3     for (int i = 0; i < count; i++) {  
4         total = total + values[i];  
5     }  
6     return total / count;  
7 }
```

LINE 3 · STEP 9 OF 10

i becomes 3; $3 < 3$ is false — the loop ends.

VARIABLES

values = {10, 20, 30}

count = 3

total = 0 → 10 → 30 → 60

i = 0 → 1 → 2 → 3

HANDS BACK

The average of an array.

```
1 float average(float values[], int count) {  
2     float total = 0;  
3     for (int i = 0; i < count; i++) {  
4         total = total + values[i];  
5     }  
6     return total / count;  
7 }
```

LINE 6 · STEP 10 OF 10

Hand back $60 / 3 = 20$.

VARIABLES

values = {10, 20, 30}

count = 3

total = 0 → 10 → 30 → 60

i = 0 → 1 → 2 → 3

HANDS BACK

20

The last 10 readings.

```
1  const int SIZE = 10;
2  float readings[SIZE];
3  int next = 0;
4
5  void loop() {
6      float light = incipe.getLightIntensity();
7      if (light >= 0) {
8          readings[next] = light;
9          next = (next + 1) % SIZE;
10         Serial.println(average(readings, SIZE));
11     }
12     delay(100);
13 }
```

LINE 1 · STEP 1 OF 13

Keep the last 10 readings.

VARIABLES

SIZE = 10

SERIAL MONITOR

The last 10 readings.

```
1  const int SIZE = 10;
2  float readings[SIZE];
3  int next = 0;
4
5  void loop() {
6      float light = incipe.getLightIntensity();
7      if (light >= 0) {
8          readings[next] = light;
9          next = (next + 1) % SIZE;
10         Serial.println(average(readings, SIZE));
11     }
12     delay(100);
13 }
```

LINE 2 · STEP 2 OF 13

Ten boxes. A global array starts as all 0.

VARIABLES

SIZE = 10

readings = {0,0,0,0,0,0,0,0,0,0}

SERIAL MONITOR

The last 10 readings.

```
1  const int SIZE = 10;
2  float readings[SIZE];
3  int next = 0;
4
5  void loop() {
6      float light = incipe.getLightIntensity();
7      if (light >= 0) {
8          readings[next] = light;
9          next = (next + 1) % SIZE;
10         Serial.println(average(readings, SIZE));
11     }
12     delay(100);
13 }
```

LINE 3 · STEP 3 OF 13

next is the box the next reading goes in.

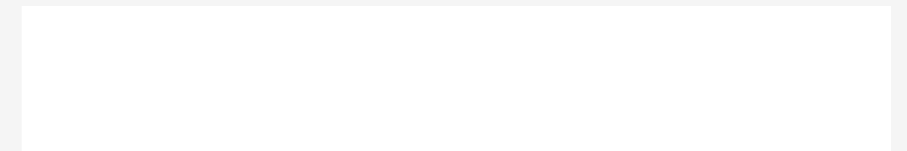
VARIABLES

SIZE = 10

readings = {0,0,0,0,0,0,0,0,0,0}

next = 0

SERIAL MONITOR



The last 10 readings.

```
1  const int SIZE = 10;
2  float readings[SIZE];
3  int next = 0;
4
5  void loop() {
6    float light = incipe.getLightIntensity();
7    if (light >= 0) {
8      readings[next] = light;
9      next = (next + 1) % SIZE;
10     Serial.println(average(readings, SIZE));
11   }
12   delay(100);
13 }
```

LINE 6 · STEP 4 OF 13

Read the light. Say it is 500.

VARIABLES

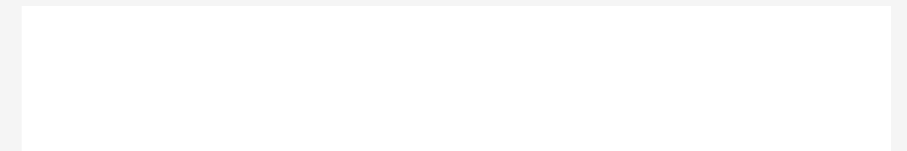
SIZE = 10

readings = {0,0,0,0,0,0,0,0,0,0}

next = 0

light = 500

SERIAL MONITOR



The last 10 readings.

```
1  const int SIZE = 10;
2  float readings[SIZE];
3  int next = 0;
4
5  void loop() {
6      float light = incipe.getLightIntensity();
7      if (light >= 0) {
8          readings[next] = light;
9          next = (next + 1) % SIZE;
10         Serial.println(average(readings, SIZE));
11     }
12     delay(100);
13 }
```

LINE 7 · STEP 5 OF 13

Not -1, so the sensor is there.

VARIABLES

SIZE = 10

readings = {0,0,0,0,0,0,0,0,0,0}

next = 0

light = 500

SERIAL MONITOR

The last 10 readings.

```
1  const int SIZE = 10;
2  float readings[SIZE];
3  int next = 0;
4
5  void loop() {
6      float light = incipe.getLightIntensity();
7      if (light >= 0) {
8          readings[next] = light;
9          next = (next + 1) % SIZE;
10         Serial.println(average(readings, SIZE));
11     }
12     delay(100);
13 }
```

LINE 8 · STEP 6 OF 13

Put it in box 0.

VARIABLES

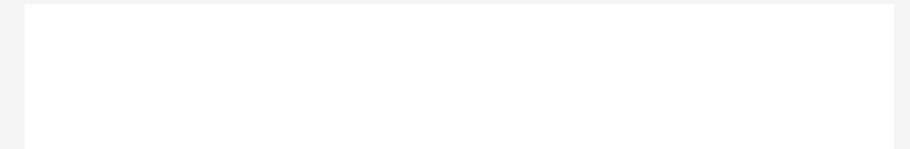
SIZE = 10

readings = {500,0,0,0,0,0,0,0,0,0}

next = 0

light = 500

SERIAL MONITOR



The last 10 readings.

```
1  const int SIZE = 10;
2  float readings[SIZE];
3  int next = 0;
4
5  void loop() {
6      float light = incipe.getLightIntensity();
7      if (light >= 0) {
8          readings[next] = light;
9          next = (next + 1) % SIZE;
10         Serial.println(average(readings, SIZE));
11     }
12     delay(100);
13 }
```

LINE 9 · STEP 7 OF 13

Move to the next box. % wraps 10 back to 0, so after box 9 it reuses box 0.

VARIABLES

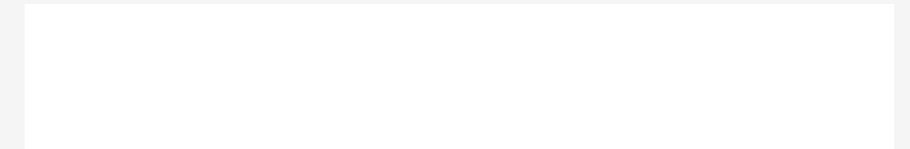
SIZE = 10

readings = {500,0,0,0,0,0,0,0,0,0}

next = 0 → 1

light = 500

SERIAL MONITOR



The last 10 readings.

```
1  const int SIZE = 10;
2  float readings[SIZE];
3  int next = 0;
4
5  void loop() {
6      float light = incipe.getLightIntensity();
7      if (light >= 0) {
8          readings[next] = light;
9          next = (next + 1) % SIZE;
10         Serial.println(average(readings, SIZE));
11     }
12     delay(100);
13 }
```

LINE 10 · STEP 8 OF 13

Average of the 10 boxes: 500 / 10.

VARIABLES

SIZE = 10

readings = {500,0,0,0,0,0,0,0,0,0}

next = 0 → 1

light = 500

SERIAL MONITOR

50.00

The last 10 readings.

```
1  const int SIZE = 10;
2  float readings[SIZE];
3  int next = 0;
4
5  void loop() {
6      float light = incipe.getLightIntensity();
7      if (light >= 0) {
8          readings[next] = light;
9          next = (next + 1) % SIZE;
10         Serial.println(average(readings, SIZE));
11     }
12     delay(100);
13 }
```

LINE 12 · STEP 9 OF 13

Wait 100 ms — inside the Wiki's 20–500 ms.

VARIABLES

SIZE = 10

readings = {500,0,0,0,0,0,0,0,0,0}

next = 0 → 1

light = 500

SERIAL MONITOR

50.00

The last 10 readings.

```
1  const int SIZE = 10;
2  float readings[SIZE];
3  int next = 0;
4
5  void loop() {
6    float light = incipe.getLightIntensity();
7    if (light >= 0) {
8      readings[next] = light;
9      next = (next + 1) % SIZE;
10     Serial.println(average(readings, SIZE));
11   }
12   delay(100);
13 }
```

LINE 6 · STEP 10 OF 13

Next pass: say the light reads 520.

VARIABLES

SIZE = 10

readings = {500,0,0,0,0,0,0,0,0,0}

next = 0 → 1

light = 500 → 520

SERIAL MONITOR

50.00

The last 10 readings.

```
1  const int SIZE = 10;
2  float readings[SIZE];
3  int next = 0;
4
5  void loop() {
6      float light = incipe.getLightIntensity();
7      if (light >= 0) {
8          readings[next] = light;
9          next = (next + 1) % SIZE;
10         Serial.println(average(readings, SIZE));
11     }
12     delay(100);
13 }
```

LINE 8 · STEP 11 OF 13

Into box 1.

VARIABLES

SIZE = 10

readings = {500, 520, 0, 0, 0, 0, 0, 0, 0, 0}

next = 0 → 1

light = 500 → 520

SERIAL MONITOR

50.00

The last 10 readings.

```
1  const int SIZE = 10;
2  float readings[SIZE];
3  int next = 0;
4
5  void loop() {
6      float light = incipe.getLightIntensity();
7      if (light >= 0) {
8          readings[next] = light;
9          next = (next + 1) % SIZE;
10         Serial.println(average(readings, SIZE));
11     }
12     delay(100);
13 }
```

LINE 9 · STEP 12 OF 13

On to box 2.

VARIABLES

SIZE = 10

readings = {500, 520, 0, 0, 0, 0, 0, 0, 0, 0}

next = 0 → 1 → 2

light = 500 → 520

SERIAL MONITOR

50.00

The last 10 readings.

```
1  const int SIZE = 10;
2  float readings[SIZE];
3  int next = 0;
4
5  void loop() {
6      float light = incipe.getLightIntensity();
7      if (light >= 0) {
8          readings[next] = light;
9          next = (next + 1) % SIZE;
10         Serial.println(average(readings, SIZE));
11     }
12     delay(100);
13 }
```

LINE 10 · STEP 13 OF 13

$(500 + 520) / 10$.

VARIABLES

SIZE = 10

readings = {500, 520, 0, 0, 0, 0, 0, 0, 0, 0}

next = 0 → 1 → 2

light = 500 → 520

SERIAL MONITOR

50.00

102.00

The first nine averages are too low.

The empty boxes still hold 0. After 10 readings every box is real and the average is right. Fixing that is part of your task.

Build it, plot it, then make it better.

Step	Do this
1	Build the sketch and open the Plotter.
2	Print the raw light and the average on one line, so the Plotter draws both: <code>Serial.print(light); Serial.print(" ");</code> then <code>println</code> the average.
3	Fix the low start: keep a count of readings so far (up to 10) and average only those.
4	Try SIZE 3 and 30. Which follows your hand faster? Which is smoother?

No & this time. What prints?

READ THE CODE

```
void addBonusCopy(int score) {  
    score = score + 10;  
}
```

```
int points = 50;  
addBonusCopy(points);  
Serial.println(points);
```

ANSWER

50

WHY

Without &, the function gets a copy. It adds 10 to the copy; points is untouched.

What prints?

READ THE CODE

```
int a = 5;  
int *p = &a;  
*p = *p * 2;  
Serial.println(a);
```

ANSWER

10

WHY

p points to a, so $*p * 2$ is $5 \times 2 = 10$, written back into a.

What are 6 & 3 and 6 | 1?

LINE UP THE BITS

110 (6)
& 011 (3)

110 (6)
| 001 (1)

ANSWER

2 and 7

WHY

110 & 011 = 010 = 2: only the middle bit is 1 in both. 110 | 001 = 111 = 7: every bit is 1 in at least one.

What is $1 \ll 4$?

SHIFT THE BITS

00001

$\ll 4$

ANSWER

16

WHY

00001 shifted left 4 places is 10000: switch 4, worth 16.

The first reading is 500. What prints, and why not 500?

THE MOVING AVERAGE

```
SIZE = 10  
readings = {500,0,0,0,0,0,0,0,0,0}  
Serial.println(average(readings, SIZE));
```

ANSWER

50.00

WHY

The other nine boxes still hold 0, so the average is $500 / 10 = 50$. It becomes right once all ten boxes are filled.

Three things to take away.

1

`int &x` is a reference — another name for the same variable, so a function can change it.

2

A pointer holds an address: `&` takes it, `*` goes to it. An array's name works like a pointer to its first box.

3

`|` sets a bit, `&` checks it, `^` flips it. A moving average keeps the last `N` readings in an array.

Next — Session 4: Structs & state machines. Group related data with struct, and build a traffic light that moves from state to state.